



Master 1 GIL — Technologies XML
**Projet : Service REST SEPA (ISO 20022) &
Déploiement Conteneurisé**

RAPPORT TECHNIQUE DE PROJET

Projet : SEPA26 — Service REST ISO 20022

Version : 1.0.0

Date : Octobre 2026

Auteurs : Umm-Habibah Ouattara & Florian Pépin

Technologies : Spring Boot 4.1.1, Java 25 LTS, Docker, MariaDB

Table des matières

1	Introduction	2
2	Contexte et Objectifs	2
3	Organisation du Projet & Stack Technique	3
3.1	Organisation du Dépôt & Arborescence	3
3.2	Stack Technique	3
4	Points d'Accès de l'Application	4
5	Usine Logicielle, Conteneurisation & CI/CD	4
5.1	Build Multi-Stage avec Docker	4
5.2	Orchestration Locale avec Docker Compose	5
5.3	Gestion des Variables d'Environnement	5
5.4	Pipeline d'Intégration Continue (GitHub Actions)	5
A	Annexe : Catalogue Exhaustif des Classes et Composants	6

1 Introduction

Ce projet a pour objectif la conception et l'implémentation d'un service web REST complet dédié à la gestion, la validation et la restitution de transactions de prélèvement bancaire SEPA, basées sur la norme internationale **ISO 20022** (pain.008.001.02).

Le service permet l'ingestion de messages XML, leur validation stricte selon un schéma XML Schema (XSD), leur stockage persistant dans une base relationnelle, leur recherche multicritères, ainsi que leur restitution sous forme de flux XML structurés ou de vues HTML conviviales transformées par feuilles de style XSLT 3.0.

L'application s'accompagne d'une infrastructure moderne et autonome basée sur **Docker**, **Docker Compose** et un pipeline d'Intégration Continue (**GitHub Actions**).

2 Contexte et Objectifs

- **Cadre académique :** Master 1 Génie de l'Informatique Logicielle (GIL) — Université de Rouen Normandie.
- **Norme métier :** Format XML ISO 20022 pour les prélèvements SEPA directs (*Customer Direct Debit Initiation*).
- **Exigences de qualité logicielle :**
 - Séparation stricte des responsabilités entre couches applicatives.
 - Inversion des dépendances et découplage via interfaces Java.
 - Élimination de la duplication de code et mutualisation des composants.
 - Déploiement portable, reproductible et sans adhérence à une machine ou à un serveur externe.

3 Organisation du Projet & Stack Technique

Le projet repose sur une séparation nette entre le code applicatif et les descripteurs d'infrastructure, garantissant un environnement modulaire, maintenable et immédiatement opérationnel.

3.1 Organisation du Dépôt & Arborescence

Afin d'isoler nettement les outils d'infrastructure du code source applicatif, le dépôt est structuré comme suit :

- **app/** : Application Spring Boot autonome :
 - **src/main/java/** : Code source Java (contrôleurs, services, entités, DTOs).
 - **src/main/resources/** : Configuration, templates Thymeleaf, CSS et ressources XML/XSD/XSLT.
 - **src/test/** : Suites de tests unitaires et d'intégration.
 - **pom.xml** : Descripteur Maven de dépendances et de plugins.
 - **checkstyle.xml** : Règles de conformité et d'analyse statique du code (linter).
 - **Dockerfile** : Fichier de build multi-stage de l'image Docker.
- **doc/** : Documentation technique et sources LaTeX du présent rapport.
- **docker-compose.yml** : Orchestration locale des conteneurs applicatif et base de données.
- **.env & .env.example** : Paramètres de configuration et variables d'environnement.
- **.github/workflows/** : Pipeline d'Intégration Continue (GitHub Actions).

3.2 Stack Technique

- **Langage** : Java 25 (LTS).
- **Framework** : Spring Boot 4.1.1 (Spring Web, Spring Data JPA).
- **Gestion des dépendances & Build** : Apache Maven.
- **Qualité de code & Linter** : Maven Checkstyle Plugin 3.6.0.
- **Base de données** : MariaDB 11 en production / Docker, et H2 Database en mémoire pour l'exécution des tests.
- **Traitement XML** : Jakarta XML Binding (JAXB 4.0), Saxon-HE 12.9 (moteur XSLT 3.0), Jackson XML.
- **Moteur de rendu Web** : Thymeleaf.
- **Conteneurisation & CI** : Docker, Docker Compose, GitHub Actions.

4 Points d'Accès de l'Application

L'application expose ses fonctionnalités à travers deux interfaces complémentaires : une API REST ISO 20022 pour les échanges de données XML, et une interface Web Thymeleaf pour la consultation interactive.

Méthode	Endpoint	Format	Description
GET	/	HTML	Page d'accueil du service
GET	/help	HTML	Documentation interactive des points d'accès
GET	/transfert	HTML	Formulaire d'importation et validation de flux XML
GET	/sepa26/resume/html	HTML	Tableau de bord des 10 dernières transactions
GET	/sepa26/resume/xml	XML	Synthèse XML des 10 derniers documents
GET	/sepa26/xml/{id}	XML	Fiche complète du document SEPA
GET	/sepa26/html/{id}	HTML	Rendu stylisé du document via feuille XSLT
POST	/sepa26/insert	XML	Insertion d'un flux SEPA (validation XSD)
DELETE	/sepa26/delete/{id}	XML	Suppression d'une transaction
GET	/sepa26/search	XML	Recherche multicritères (date, montant)

TABLE 1 – Synthèse des points d'accès de l'application SEPA26

5 Usine Logicielle, Conteneurisation & CI/CD

Pour garantir un déploiement fiable et reproductible sur n'importe quel environnement, le projet s'appuie sur une infrastructure conteneurisée complète.

5.1 Build Multi-Stage avec Docker

Le fichier `Dockerfile` adopte une approche multi-stage optimisant à la fois la sécurité et la légèreté de l'image finale :

1. **Étape de compilation (*Builder*) :**

S'appuie sur l'image `eclipse-temurin:25-jdk-alpine`. Elle intègre Maven, compile les sources et génère l'archive exécutable `sepa26server.jar`.

2. **Étape d'exécution (*Runtime*) :**

S'appuie sur l'image minimale `eclipse-temurin:25-jre-alpine`. Seul le binaire compilé y est injecté. L'application est exécutée sous un compte dédié sans privilège (`spring:spring`) pour une sécurité optimale.

L'image produite est baptisée `sepa26-app:latest`.

5.2 Orchestration Locale avec Docker Compose

Le fichier `docker-compose.yml` orchestre les deux services nécessaires au fonctionnement complet du projet :

- **Service db** : Conteneur MariaDB 11 avec volume persistant (`mariadb_data`) et contrôle de santé (*healthcheck*).
- **Service app** : Conteneur applicatif `sepa26-app` exposé sur le port 8100, démarré automatiquement une fois la base de données saine et prête à recevoir des connexions.

Le lancement s'effectue en une commande unique :

```
docker compose up -build
```

5.3 Gestion des Variables d'Environnement

La configuration de l'infrastructure est externalisée via des fichiers d'environnement :

- `.env.example` : Modèle versionné décrivant les variables nécessaires (`SERVER_PORT`, `DB_URL`, `DB_USER`, `DB_PASSWORD`, `DB_ROOT_PASSWORD`).
- `.env` : Fichier de configuration local, ignoré par Git pour préserver la confidentialité des paramètres.

5.4 Pipeline d'Intégration Continue (GitHub Actions)

Un workflow GitHub Actions automatisé (`.github/workflows/ci.yml`) valide l'intégrité du code à chaque *push* et *pull request* :

1. **Vérification du code (*Checkstyle*)** : Exécution du linter (`mvn checkstyle:check`) pour valider les règles de nommage et l'absence d'imports superflus.
2. **Compilation et tests (*Build & Test*)** : Mise en place de Java 25 (Temurin) avec cache Maven et exécution des tests unitaires (`mvn clean test`).
3. **Validation du conteneur (*Docker Build*)** : Construction de l'image Docker multi-stage à partir du contexte applicatif (`docker build -t sepa26-app:latest ./app`).

A Annexe : Catalogue Exhaustif des Classes et Composants

Ce catalogue détaille l'ensemble des classes, interfaces et records développés au sein du projet SEPA26, avec leur localisation, leur type et leur responsabilité métier.

Composant / Type	Rôle et Responsabilité
Point d'Entrée Principal (fr.univrouen.sepa26)	
Sepa26Application <i>Classe d'application</i>	Point d'entrée exécutable démarrant le contexte Spring Boot.
Contrôleurs REST et Web (fr.univrouen.sepa26.controllers)	
SepaApiController <i>Contrôleur REST</i>	API REST (@RestController) gérant l'ensemble des flux XML (/sepa26) : consultation, insertion avec validation XSD, suppression et recherche multicritères.
SepaWebController <i>Contrôleur Web</i>	Contrôleur Web (@Controller) gérant l'interface HTML Thymeleaf : accueil, documentation, transfert, tableau récapitulatif et restitution XSLT.
Services Métier et Traitement XML (fr.univrouen.sepa26.services)	
SepaService <i>Service Métier</i>	Façade métier coordonnant la persistance JPA, le contrôle d'unicité des virements et la délégation aux services XML.
XmlValidationService <i>Interface</i>	Contrat du service de validation des flux XML selon les schémas XSD.
XmlValidationServiceImpl <i>Implémentation</i>	Validation XSD avec mise en cache thread-safe du schéma sepa26.xsd.
XmlSerializationService <i>Interface</i>	Contrat du service de sérialisation et désérialisation XML JAXB.
XmlSerializationServiceImpl <i>Implémentation</i>	Sérialisation JAXB avec mise en cache du JAXBContext réutilisable.
XsltTransformationService <i>Interface</i>	Contrat du service de transformation de documents XML en HTML via XSLT.
EndpointDocumentation-Service <i>Interface</i>	Contrat fournissant les métadonnées de documentation des points d'accès.
EndpointDocumentation-ServiceImpl <i>Implémentation</i>	Documentation descriptive exhaustive des dix points d'accès pour la page d'aide.
Utilitaires et Adaptateurs (fr.univrouen.sepa26.util)	
XsltTransformer <i>Composant Saxon</i>	Moteur Saxon-HE appliquant la feuille de style sepa26.xslt sur les documents.
LocalDateAdapter <i>Adaptateur JAXB</i>	Conversion bidirectionnelle entre les dates ISO (YYYY-MM-DD) et LocalDate.
LocalDateTimeAdapter <i>Adaptateur JAXB</i>	Conversion bidirectionnelle des horodatages ISO et LocalDateTime.

Composant / Type	Rôle et Responsabilité
Persistance des Données (fr.univrouen.sepa26.repository)	
DocumentRepository <i>Interface Spring Data JPA</i>	Interface de persistance assurant les opérations CRUD, la recherche par plage de dates et montants, et la vérification par PmtId.
Initialisation (fr.univrouen.sepa26.init)	
DataInitializer <i>Composant CommandLineRunner</i>	Insertion automatique de deux documents types au démarrage si la base est vide.
Modèle Métier ISO 20022 (fr.univrouen.sepa26.model)	
Document <i>Entité Racine JPA/JAXB</i>	Entité racine représentant le document ISO 20022 complet.
CstmrDrctDbtInitn <i>Sous-entité Document</i>	Bloc d'initiation du prélèvement SEPA direct.
GrpHdr <i>Sous-entité Document</i>	En-tête de lot (identifiant, horodatage, total de contrôle, nombre d'opérations).
PmtInf <i>Sous-entité Document</i>	Informations de paiement regroupant créancier, date d'échéance et transactions.
DrctDbtTxInf <i>Sous-entité Document</i>	Transaction unitaire élémentaire (débitteur, montant, mandat et compte).
Objets de Transfert DTO (fr.univrouen.sepa26.dto)	
ValidationResult <i>Record DTO</i>	Record immutable contenant l'état de validation XSD et l'erreur associée.
ParseResult <i>Record DTO</i>	Record générique encapsulant le résultat d'une désérialisation XML JAXB.
SepaResponse <i>DTO XML</i>	Réponse XML standardisée pour les écritures (INSERTED, DELETED, ERROR).
DocumentList <i>DTO XML</i>	DTO conteneur XML pour la restitution de listes de documents.
SearchResponse <i>DTO XML</i>	Réponse XML de recherche multicritères (OK, NONE, ERROR).